

Name: Sanidhya Negi

Sap Id: 590022442

Batch :19

WEEK 3

DAA LAB

Q1 objective-. Take a random input of thousand numbers and perform the following algorithm

- 1.Time comparison of linear and binary search.
- 2.comparison of bubble sort, insertion sort, quick sort, merge sort.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define N 1000
```

```
// Function for Linear Search
```

```
int linearSearch(int a[], int n, int key)
```

```
{
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        if (a[i] == key)
```

```
            return i;
```

```
    }
```

```
    return -1;
```

```
}
```

```
// Function for Binary Search
```

```
int binarySearch(int a[], int n, int key)
```

```
{
```

```
    int low = 0, high = n - 1;
```

```
    while (low <= high)
```

```
    {
```

```
        int mid = (low + high) / 2;
```

```

    if (a[mid] == key)
        return mid;

    else if (a[mid] < key)
        low = mid + 1;
    else
        high = mid - 1;
}

return -1;
}

// Bubble Sort
void bubbleSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

```

```
    }  
}
```

// Insertion Sort

```
void insertionSort(int a[], int n)
```

```
{  
    for (int i = 1; i < n; i++)  
    {  
        int key = a[i];  
        int j = i - 1;  
  
        while (j >= 0 && a[j] > key)  
        {  
            a[j + 1] = a[j];  
            j--;  
        }  
  
        a[j + 1] = key;  
    }  
}
```

// Quick Sort

```
void quickSort(int a[], int low, int high)
```

```
{  
    if (low < high)  
    {
```

```
int pivot = a[high];
```

```
int i = low - 1;
```

```
for (int j = low; j < high; j++)
```

```
{
```

```
    if (a[j] < pivot)
```

```
    {
```

```
        i++;
```

```
        int temp = a[i];
```

```
        a[i] = a[j];
```

```
        a[j] = temp;
```

```
    }
```

```
}
```

```
int temp = a[i + 1];
```

```
a[i + 1] = a[high];
```

```
a[high] = temp;
```

```
int p = i + 1;
```

```
quickSort(a, low, p - 1);
```

```
quickSort(a, p + 1, high);
```

```
}
```

```
}
```

```
// Merge function

void merge(int a[], int low, int mid, int high)
{
    int temp[N];

    int i = low;
    int j = mid + 1;
    int k = low;

    while (i <= mid && j <= high)
    {
        if (a[i] < a[j])
        {
            temp[k] = a[i];
            i++;
        }
        else
        {
            temp[k] = a[j];
            j++;
        }

        k++;
    }

    while (i <= mid)
    {
```

```
    temp[k] = a[i];  
    i++;  
    k++;  
}
```

```
while (j <= high)  
{  
    temp[k] = a[j];  
    j++;  
    k++;  
}
```

```
for (i = low; i <= high; i++)  
{  
    a[i] = temp[i];  
}  
}
```

// Merge Sort

```
void mergeSort(int a[], int low, int high)  
{  
    if (low < high)  
    {  
        int mid = (low + high) / 2;  
  
        mergeSort(a, low, mid);
```

```
mergeSort(a, mid + 1, high);
```

```
merge(a, low, mid, high);
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
int original[N];
```

```
int a[N];
```

```
int key;
```

```
clock_t start, end;
```

```
double time_taken;
```

```
// Generate 1000 random numbers
```

```
srand(time(NULL));
```

```
for (int i = 0; i < N; i++)
```

```
{
```

```
    original[i] = rand() % 10000;
```

```
}
```

```
printf("1000 random numbers generated.\n\n");
```

```
// ----- SEARCHING -----
```

```
key = original[500];

// Linear Search
start = clock();

linearSearch(original, N, key);

end = clock();

time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

printf("Linear Search Time = %f seconds\n", time_taken);

// First sort the array for Binary Search
for (int i = 0; i < N; i++)
{
    a[i] = original[i];
}

quickSort(a, 0, N - 1);

// Binary Search
start = clock();

binarySearch(a, N, key);
```

```
end = clock();
```

```
time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
printf("Binary Search Time = %f seconds\n", time_taken);
```

```
// ----- SORTING -----
```

```
// Bubble Sort
```

```
for (int i = 0; i < N; i++)
```

```
    a[i] = original[i];
```

```
start = clock();
```

```
bubbleSort(a, N);
```

```
end = clock();
```

```
time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
printf("Bubble Sort Time = %f seconds\n", time_taken);
```

```
// Insertion Sort
```

```
for (int i = 0; i < N; i++)
```

```
    a[i] = original[i];
```

```
start = clock();
```

```
insertionSort(a, N);
```

```
end = clock();
```

```
time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
printf("Insertion Sort Time = %f seconds\n", time_taken);
```

```
// Quick Sort
```

```
for (int i = 0; i < N; i++)
```

```
    a[i] = original[i];
```

```
start = clock();
```

```
quickSort(a, 0, N - 1);
```

```
end = clock();
```

```
time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
printf("Quick Sort Time    = %f seconds\n", time_taken);
```

```
// Merge Sort
```

```
for (int i = 0; i < N; i++)  
    a[i] = original[i];  
  
start = clock();  
  
mergeSort(a, 0, N - 1);  
  
end = clock();  
  
time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;  
  
printf("Merge Sort Time    = %f seconds\n", time_taken);  
  
return 0;  
}
```

Code for graph:

```
import random

import time

import matplotlib.pyplot as plt


# Number of elements

N = 1000


# Generate 1000 random numbers

original = [random.randint(1, 10000) for _ in range(N)]


# -----

# SEARCHING ALGORITHMS

# -----


# Linear Search

def linear_search(arr, key):

    for i in range(len(arr)):

        if arr[i] == key:

            return i

    return -1


# Binary Search

def binary_search(arr, key):

    low = 0
```

```
high = len(arr) - 1
```

```
while low <= high:
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == key:
```

```
        return mid
```

```
    elif arr[mid] < key:
```

```
        low = mid + 1
```

```
    else:
```

```
        high = mid - 1
```

```
return -1
```

```
# -----
```

```
# SORTING ALGORITHMS
```

```
# -----
```

```
# Bubble Sort
```

```
def bubble_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n - 1):
```

```
        for j in range(n - i - 1):
```

```
            if arr[j] > arr[j + 1]:
```

```
arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
# Insertion Sort
```

```
def insertion_sort(arr):
```

```
    for i in range(1, len(arr)):
```

```
        key = arr[i]
```

```
        j = i - 1
```

```
        while j >= 0 and arr[j] > key:
```

```
            arr[j + 1] = arr[j]
```

```
            j -= 1
```

```
        arr[j + 1] = key
```

```
# Quick Sort
```

```
def quick_sort(arr):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    pivot = arr[-1]
```

```
    left = []
```

```
    right = []
```

```
for x in arr[:-1]:
    if x <= pivot:
        left.append(x)
    else:
        right.append(x)

return quick_sort(left) + [pivot] + quick_sort(right)
```

Merge Sort

```
def merge_sort(arr):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid])
```

```
    right = merge_sort(arr[mid:])
```

```
    result = []
```

```
    i = 0
```

```
    j = 0
```

```
    while i < len(left) and j < len(right):
```

```
if left[i] < right[j]:  
    result.append(left[i])  
    i += 1  
else:  
    result.append(right[j])  
    j += 1
```

```
result.extend(left[i:])  
result.extend(right[j:])
```

```
return result
```

```
# -----  
# SEARCH TIME COMPARISON  
# -----
```

```
key = original[500]
```

```
# Linear Search
```

```
start = time.perf_counter()  
linear_search(original, key)  
linear_time = time.perf_counter() - start
```

```
# Binary Search needs sorted array
```

```
sorted_array = sorted(original)
```

```
start = time.perf_counter()
binary_search(sorted_array, key)
binary_time = time.perf_counter() - start
```

```
print("SEARCHING")
print("-----")
print("Linear Search :", linear_time)
print("Binary Search :", binary_time)
```

```
# -----
# SORTING TIME COMPARISON
# -----
```

```
# Bubble Sort
arr = original.copy()
```

```
start = time.perf_counter()
bubble_sort(arr)
bubble_time = time.perf_counter() - start
```

```
# Insertion Sort
arr = original.copy()
```

```
start = time.perf_counter()
insertion_sort(arr)
insertion_time = time.perf_counter() - start
```

```
# Quick Sort
```

```
arr = original.copy()
```

```
start = time.perf_counter()
quick_sort(arr)
quick_time = time.perf_counter() - start
```

```
# Merge Sort
```

```
arr = original.copy()
```

```
start = time.perf_counter()
merge_sort(arr)
merge_time = time.perf_counter() - start
```

```
print("\nSORTING")
print("-----")
print("Bubble Sort   :", bubble_time)
print("Insertion Sort :", insertion_time)
print("Quick Sort    :", quick_time)
```

```
print("Merge Sort    :", merge_time)
```

```
# -----
```

```
# GRAPH 1 - SEARCHING
```

```
# -----
```

```
search_names = ["Linear Search", "Binary Search"]
```

```
search_times = [linear_time, binary_time]
```

```
plt.figure(figsize=(8, 5))
```

```
plt.bar(search_names, search_times)
```

```
plt.xlabel("Searching Algorithm")
```

```
plt.ylabel("Time (seconds)")
```

```
plt.title("Linear Search vs Binary Search")
```

```
plt.show()
```

```
# -----
```

```
# GRAPH 2 - SORTING
```

```
# -----
```

```
sort_names = [
```

```
    "Bubble Sort",
```

```
"Insertion Sort",  
"Quick Sort",  
"Merge Sort"  
]
```

```
sort_times = [  
    bubble_time,  
    insertion_time,  
    quick_time,  
    merge_time  
]
```

```
plt.figure(figsize=(9, 5))  
plt.bar(sort_names, sort_times)
```

```
plt.xlabel("Sorting Algorithm")  
plt.ylabel("Time (seconds)")  
plt.title("Sorting Algorithm Comparison")
```

```
plt.show()
```

Output:

1000 random numbers generated.

SEARCHING ALGORITHMS

Linear Search Time = 0.142000 seconds

Binary Search Time = 0.000000 seconds

SORTING ALGORITHMS

Bubble Sort Time = 0.042000 seconds

Insertion Sort Time = 0.018000 seconds

Quick Sort Time = 0.010000 seconds

Merge Sort Time = 0.010000 seconds

SEARCHING

Linear Search : 2.648...e-05

Binary Search : 5.999...e-06

SORTING

Bubble Sort : 0.064...

Insertion Sort : 0.026...

Quick Sort : 0.001...

Merge Sort : 0.004...